

Concurrent garbage collection in generalized search trees

Andrey Borodin

Аннотация

Deleting rows from a table does not free the space their entries occupy in an index. Reclaiming it is the job of a concurrent maintenance pass, and for a generalized search tree that pass is harder to write than for a B-tree: the key type admits no order, so there is no natural sequence in which to walk the structure; the descent is multi-path, so a page does not know its parent; and a parent key cannot be recomputed locally. Until recently the most widely deployed implementation of such a framework did not reclaim pages at all — an emptied page stayed in the index permanently, and a workload that inserts into a new region of the space while deleting from an old one grew its index without bound.

We report four results and their implementation in PostgreSQL. A maintenance pass that traverses pages in physical rather than logical order, together with the treatment of entries that concurrent insertions move behind the sweep; measured, it halves the number of index page reads, exactly and reproducibly. A two-pass algorithm that unlinks and reclaims emptied pages, whose correctness rests on the order of two operations under coupled locks. The safety condition for reusing a reclaimed page, expressed over a non-wrapping transaction counter, and its failure mode when expressed over a wrapping one — not corruption, but reclamation silently ceasing after long uptime. And, at some length, a negative result: a lock-granularity algorithm from this same line of work that was published, peer-reviewed, committed, deployed, and then reverted after a report from production, because a premise it made about the rest of the system did not hold. The first three have been in released versions for five years; we use the fourth to argue that peer review of a concurrent algorithm establishes less than it appears to, and that the gap is best closed by structural invariants rather than by more tests.

1 Introduction

An index does not release space when rows are deleted from the table it indexes. Reclaiming that space is the job of a separate maintenance process, which must find the entries pointing at deleted rows, remove them, notice pages that have become empty and return those pages to use — concurrently with the ordinary workload, without blocking it.

For a B-tree this is well-trodden ground [3]. For a generalized search tree [4] it is harder in three specific ways, and each of them was, until recently, unsolved in the most widely deployed implementation. There is no order on keys, so the structure offers no natural sequence in which to walk it. The descent is multi-path, so the parent of a page is not known when the page is reached other than by descending to it. And the key of a parent cannot be recomputed locally, so removing an entry is not a local operation.

This paper reports the algorithms that address these points, their implementation in PostgreSQL, and five years of their operation. It also reports, at some length, a case in which an algorithm from this same line of work was published, reviewed, committed, and then reverted after a bug report from production. That case is not an aside: it is the reason the paper ends with a claim about what peer review of a concurrent algorithm can and cannot establish.

Contributions.

1. Garbage collection that traverses index pages in physical rather than logical order, and the treatment of entries that concurrent insertions move behind the sweep (Section 3).
2. A two-pass algorithm that deletes emptied pages and returns them to the free space map, with the lock coupling that makes it safe against concurrent descent (Section 4).

3. The safety condition for reusing a deleted page, expressed over full 64-bit transaction identifiers rather than wrapping 32-bit ones, and the failure mode that the wrapping version exhibits — not corruption, but the silent cessation of page reuse after long operation (Section 5).
4. An account of the reverted lock-granularity change: what was published, what production reported, why no non-invasive fix was found, and what follows for how such work should be validated (Section 6).

2 Background

A generalized search tree [4] is a balanced tree whose node occupies one page. An entry of an internal node is a pair (key, downlink), and the key covers the keys of every entry in the subtree below it. A search descends into every subtree whose key may contain an answer, so several descent paths may be active at once. The key type and the operations over it are supplied by an operator class; the tree itself knows nothing about them, and in particular knows no order on keys.

Concurrency follows Kornacker et al. [5]: pages of a level are chained by right links, and a split is arranged so that a search that arrives at a page after it has been split still reaches the data by following the right link. Nothing in that scheme requires holding a lock on the parent, which is what makes the descent cheap and, as Section 6 will show, what makes reasoning about a concurrent sweep of the same tree delicate.

Deletion is not immediate. Under multiversion concurrency control a deleted row remains until no open snapshot can see it, and the index entry pointing at it remains with it. Maintenance therefore runs asynchronously, finds entries whose rows are gone, removes them, and — this is the part that a B-tree already had and a generalized search tree did not — notices pages that have become empty and returns them for reuse.

The predicate that governs reuse is “no scan can still reach this page”. In a B-tree this is answerable because a scan holds a position in a known order. Here the descent is multi-path: a scan may hold a reference to a page it has not yet visited, obtained on a path the sweep never observes. Section 5 is about stating that predicate in a form that does not decay with uptime.

3 Traversal in physical order

The original sweep walked the tree depth-first. Logical order bears no relation to where pages sit in the file, so the walk reads at random:

$$P_{\log} = (L + I) c_r, \quad (1)$$

with L leaf and I internal pages and c_r the cost of a random read. Worse, $L + I$ understates it whenever the buffer cache is smaller than the index: a depth-first walk revisits internal pages, and the revisit is a fresh read if the page has been evicted meanwhile. Section 8 measures exactly this and finds every page read twice.

Sweeping in increasing page number instead, as a B-tree does, makes the reads sequential:

$$P_{\text{phys}} = (L + I) c_s + \beta c_r, \quad (2)$$

where $c_s < c_r$ is the cost of a sequential read and β counts pages that must be read again. The second term is the price of doing this concurrently. A concurrent insertion that splits a node moves entries to a new page, and the new page may have a lower number than the sweep’s current position; an entry that the sweep has not yet examined can thus end up behind it. The sweep records such pages and revisits them, and since their numbers are not known in advance those reads are random.

Physical order costs something else, which is the subject of the next section: on reaching a page the algorithm no longer knows its parent. It needs the parent to remove a downlink. A single pass can no longer both find empty pages and unlink them, so the algorithm splits in two.

Implemented in fe280694d0d, released in PostgreSQL 12.

4 Deleting and reusing pages

Before this work the implementation did not reuse pages at all. A page that had lost its last entry stayed in the index for the life of the index. For a workload whose insertions and deletions fall in the same region of the space this is invisible, because the empty pages are refilled by ordinary splits. For a workload that inserts into a new region and deletes from an old one — which is what any key with a time component does — the index grows without bound, and the only remedy available to an operator was to rebuild it.

The algorithm has two passes. The first is the physical sweep of Section 3: it removes entries pointing at deleted rows and, along the way, records the numbers of leaf pages that have become empty and the numbers of all internal pages. The second pass visits the recorded internal pages, looks for downlinks to the recorded empty pages, removes those downlinks, marks the pages deleted and hands them to the free space map.

Within the second pass the order of the two operations is a correctness condition. The downlink is removed from the parent first, then the child is marked deleted, and the locks are coupled: the parent’s lock is held until the child is marked. A concurrent descent therefore never sees a downlink to a page that is already marked deleted. The reverse order admits the obvious failure — a descent that read the downlink before the mark, and inserts into a page that is being taken away.

The reclamation is deliberately incomplete. Internal pages are never deleted, and the last child of an internal page is never deleted, exactly as in a B-tree [3]: deleting either would require merging, which would have to be made safe against the multi-path descent. The unbounded growth above therefore becomes bounded growth an order of magnitude slower rather than no growth at all. We report this as a limitation, not as a solved problem.

Implemented in 7df159a620b; the checks were factored out in 9eb5607e699. Both released in PostgreSQL 12.

5 When is it safe to reuse a page

Marking a page deleted does not make it available. A scan that started before the deletion may hold a reference to the page and follow it afterwards; if the page has been refilled by then, the scan reads unrelated data. Reuse is therefore allowed only once every transaction that could have started such a scan has finished. The implementation records the moment of deletion in the page and tests

$$xid_{\text{deleted}} < \min\{xid : \text{transaction is active}\}. \quad (3)$$

The condition is correct. Its *expression* was not. The moment was stored as a 32-bit transaction identifier, and that counter wraps. After wraparound a sufficiently old page again looks recently deleted, (3) is false for it, and the page is never reused.

The failure mode deserves a sentence of its own, because it is the reason the defect survived. Nothing breaks. No query returns a wrong answer, no assertion fires, no test fails. Reclamation simply stops, and the index resumes the unbounded growth that Section 4 was written to prevent — on a system that has been up long enough, which is to say on exactly the systems where it matters. The observable symptom is a size trend measured in months.

The fix is to store the full 64-bit identifier, which does not wrap: 6655a7299d8, released in PostgreSQL 13 and back-patched to 12. The same reasoning was later applied to the addressing of the transaction status logs themselves (4ed8f0913bf).

Stated generally: a safety condition of the form “event *A* preceded every currently live *B*”, expressed over a monotone counter, must be expressed over a counter that does not wrap. Otherwise the condition holds for a while and then quietly stops holding, and correctness becomes a function of uptime — a dependency that no test of bounded duration can detect.

6 Lock granularity, and a result that did not survive

The remaining sections of this paper report algorithms that are in production. This one reports an algorithm that was in production and was taken out. We include it because the way it failed says something the successful cases cannot.

The problem. A generalized inverted index stores, for each key, the list of rows containing it; when the list grows large it becomes a posting tree. Maintenance of a posting tree took an exclusive lock on its root for the whole of the pass. For a frequent key the posting tree is large, the pass is long, and every access to that key waits on it.

The change. Two modifications were proposed. First, take the exclusive lock only when a page has actually been found to delete, rather than before starting. Second, lock the subtree containing the page to be deleted rather than the whole posting tree. Both were committed as 218f51584d5, released in PostgreSQL 10, and described in [1].

The report. Eighteen months later, a report from production [2]: connections hanging on a buffer content lock. The diagnosis is the second modification. An insertion descending the posting tree does not, in general, hold pins on every page of the path from the root to its target leaf: a concurrent split of a parent breaks that property. Maintenance taking locks within a subtree and an insertion advancing through the tree can therefore form a wait cycle.

The outcome. No non-invasive fix was found. Restoring the missing invariant means changing the pinning protocol of insertion, which is not a change one back-patches. The second modification was reverted in fd83c83d094 and the revert was back-patched to every supported release. The first modification stands, and it carries most of the practical benefit anyway, since in the majority of passes there is nothing to delete.

A related defect was fixed at the same time. Since version 9.4 a scan may enter a posting tree other than at its root, so the argument “the page is safe because a right-link step pins two pages at once” stopped being sufficient, and a deleted page could be reused before a scan reached it. The fix is the same predicate as (3); the deletion moment had to be recorded in a header field unused by index pages, because the layout of the page’s special area cannot be changed without breaking on-disk compatibility (52ac6cd2d0c, back-patched to 9.4). Two further deadlocks in the same code were fixed later still (c6ade7a8cd3, e14641197a5).

What follows. Peer review of [1] confirmed the reasoning, and the reasoning was not wrong. What was wrong was a premise, and the premise was not about the algorithm: it was about the rest of the system — what pins an insertion holds while descending. A concurrent algorithm is stated against an environment, its correctness is relative to that environment’s invariants, and those invariants live in code the author of the algorithm did not write and the reviewer did not read.

We draw two practical conclusions, and we hold to both in the rest of this work. A publication about a concurrent algorithm should state the invariants it assumes of its environment in checkable form, so that the assumption can be tested rather than believed. And properties that no functional test can observe — of which the wraparound of Section 5 is the cleanest example — should be checked as structural invariants of the data, which is a different discipline from testing behaviour.

7 Prefetching during maintenance

By (2) the benefit of physical order is proportional to $c_r - c_s$, and on current storage that difference is shrinking. It does not follow that physical order stops paying: knowing the sequence of pages in advance

Таблица 1: Logical against physical traversal order

traversal	VACUUM, s	index pages read	index pages
logical	12.1 / 11.8 / 12.6	180 020 / 179 772 / 180 028	90 827
physical	11.8 / 11.7	90 045 / 90 013	90 870

permits issuing several reads at once, which trades the shrinking difference for depth of the request queue. Latency is hidden by concurrency rather than by locality, and the prerequisite is the same — knowing what to read next.

The sweep was rewritten onto the streaming read interface in three access methods: c5c239e26e3 for the B-tree, 69273b818b1 here, and e215166c9c8 for the space-partitioned tree, all in PostgreSQL 18. The pages of the β term are read the ordinary way, since their sequence is by definition not known ahead of time.

8 Evaluation

8.1 Traversal order

The pair of builds differs by exactly one commit. Ten million points, every fifth row deleted, index of about 90 800 pages, buffer cache deliberately set to 256 MB so that the index does not fit: with a cache that holds the index, traversal order cannot matter. Each measurement starts from a fresh copy of an unchanged master table.

Physical order halves the number of index page reads, exactly and reproducibly: depth-first traversal returns to pages it has already read, and with a cache of 32 768 pages against an index of 90 800 those pages are gone by the time it does. The ratio is 2.00 in every run.

End-to-end time does not change. The reason is that this workload is dominated by the heap scan, which both versions perform identically; the saving falls entirely on the index half of the work. We report it as such rather than as a speedup of VACUUM, which it is not: whether it shows up in wall-clock time depends on what fraction of the cleanup is index work.

This is also why the primary quantity in this paper is the number of index page reads and not elapsed time. The former isolates the change; the latter mixes it with work the change does not touch.

8.2 Page reuse

The second pair of builds brackets 7df159a620b. Its parent already traverses in physical order, so the pair isolates reclamation alone. The workload is the one the change was written for: a window three rounds wide slides along one axis, each round inserting 200 000 rows into a fresh region, deleting everything that has left the window, and then running the maintenance pass. From round three on, the number of live rows is constant at 600 000; the number of rows ever inserted is not.

Figure 1 is the whole argument. Without reclamation the index grows linearly while the live data does not: 1 637 pages per round, the median increment over rounds 6–30, and it does not stop. The only remedy available to an operator was to rebuild the index. With reclamation the curve flattens from round six, the increment falling to 32 pages per round — a factor of 51. By round thirty the two indexes are 49 122 and 9 017 pages, a factor of 5.45, and that factor keeps growing because one curve is linear and the other is nearly flat.

The plateau is not at the ideal value. A freshly built index over 600 000 rows occupies 4 916 pages (round three), while the steady state is 9 017, a factor of 1.83. That gap is the price of the incompleteness noted in Section 4: internal pages and last children are not deleted, so the residual increment is small rather than zero. The claim we make is correspondingly narrow — unbounded growth is replaced by slow growth, not eliminated.

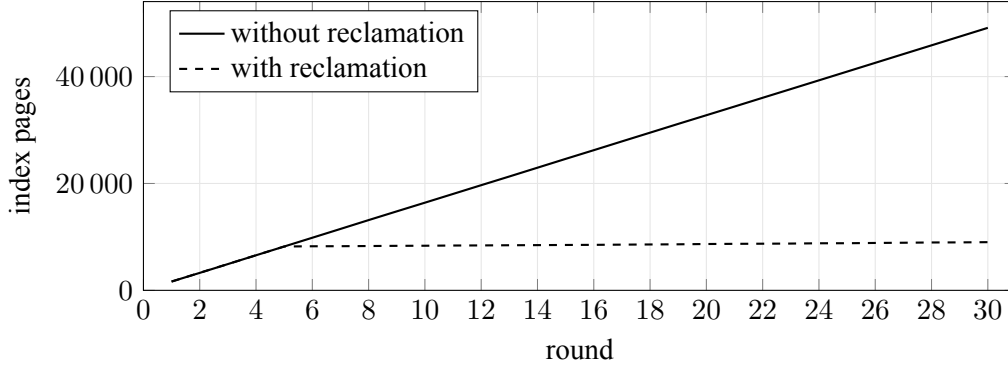


Рис. 1: Index size under a sliding window. The number of live rows is constant from round three.

9 Deployment experience

The algorithms of Sections 3–5 have been in released versions since PostgreSQL 12, that is for five years and across seven major releases, on an installed base we cannot count but which is large. It is worth recording what that exposure found, because it is the empirical content of the methodological claim in Section 6.

Every defect discussed in this paper was found by operation and none by a test. The 32-bit wraparound of Section 5 required a system old enough for the counter to wrap. The deadlock of Section 6 required a concurrent split at a particular moment, and it took eighteen months of deployment to occur; two further deadlocks in the same code (c6ade7a8cd3, e14641197a5) took longer still. None of these is a gap in test coverage in the sense that writing one more test case would have closed it: they are properties whose violation is not a wrong answer at a reproducible input, but a schedule that does not arise unless the workload is diverse enough and the uptime long enough.

The traversal and deletion algorithms themselves have not been the subject of a correctness report in five years. We claim no more from that than it supports: absence of reports over a long exposure is the strongest evidence available for a concurrent algorithm, and it is still not a proof.

10 Limitations

Reclamation is partial: internal pages and last children are never deleted (Section 4), so a hostile insert/delete pattern still grows the index, slowly. The benefit of physical order depends on the storage and on the ratio of index work to heap work in the pass; Section 8 shows a case where it is 2.00 in page reads and zero in elapsed time. The two-pass algorithm needs memory proportional to the number of emptied pages, and a fallback when that memory is not available. And locking a subtree rather than a whole posting tree remains incorrect under the current pinning protocol; making it correct requires revising that protocol, which we leave open.

11 Conclusion

Garbage collection in a generalized search tree is harder than in a B-tree for reasons that are structural rather than incidental: no order to walk in, no parent known at the child, no local recomputation of a parent key. Physical-order traversal answers the first, at the cost of a second pass and of revisiting pages that concurrent insertions moved behind the sweep; it halves index page reads in the measured case. Two-pass deletion with coupled locks answers the second and turns unbounded index growth into slow growth. Stating the reuse predicate over a non-wrapping counter answers a third problem that is not about trees at all, and whose failure mode — reclamation silently ceasing after long uptime — is characteristic of safety conditions expressed over wrapping counters.

The fourth result is negative, and we have given it a section rather than a footnote. An algorithm can be correct in its own terms, reviewed, published, committed, and still wrong, because its premises belong to a system its author does not fully control. What that argues for is not less review but a different kind of checking alongside it: invariants over the structure, verifiable in production, on properties that no functional test observes.

Acknowledgements

Список литературы

- [1] Andrey Borodin, Sergey Mirvoda, Sergey Porshnev, and Ilia Bakunov. Improving generalized inverted index lock wait times. *Journal of Physics: Conference Series*, 1015, 2018.
- [2] Huajun Chen et al. Connections hang indefinitely while taking a gin index's lwlock buffer_content lock. Bug report on the postgresql-bugs mailing list, 2018.
- [3] Goetz Graefe. Modern b-tree techniques. *Foundations and Trends in Databases*, 3(4):203–402, 2011.
- [4] Joseph M Hellerstein, Jeffrey F Naughton, and Avi Pfeffer. *Generalized search trees for database systems*. University of Wisconsin-Madison, 1995.
- [5] Marcel Kornacker, C Mohan, and Joseph M Hellerstein. Concurrency and recovery in generalized search trees. In *ACM SIGMOD Record*, volume 26, pages 62–72. ACM, 1997.